MODULE: PW5 // CUSTOM AI INTEGRATION

TRIC
TECHNOLOGY OF ROBOTICS AND IOT CENTRE

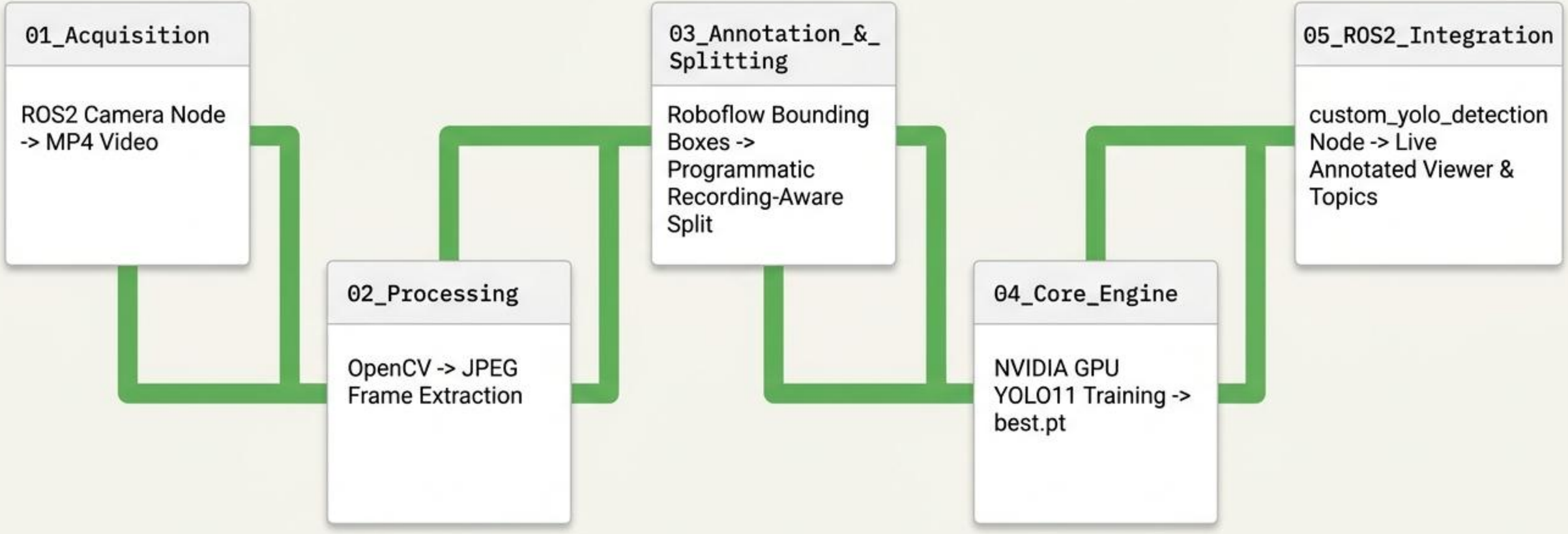
Deploying YOLO11 within a ROS2 Architecture

MindSpark Training Module: End-to-End Custom AI Integration Pipeline

- Ubuntu 24.04
- ROS2 Jazzy
- OpenCV
- Roboflow
- YOLO11

Gemini Notebook

SYSTEM ARCHITECTURE FLOWCHART



Gemini Notebook

OBSERVATION: /video_recorder output

```
$ ros2 node list
/video_recorder_node
/camera_publisher

$ ros2 topic echo /video_frames | head -n 10
---
header:
  stamp:
    sec: 1678886412
    nanosec: 567890123
  frame_id: "camera_optical_frame"
height: 1080
width: 1920
encoding: "rgb8"
is_bigendian: 0
step: 5760
data: [ ... ]

$ grep "Frames Saved" /var/log/ros/video_recorder.log
[INFO] [1678886415.123456] [video_recorder_node]:
Frames Saved: 115 (approx. 7.68 fps)
[INFO] [1678886430.567890] [video_recorder_node]:
Frames Saved: 230 (approx. 7.67 fps)
```

Hardware Reality Check

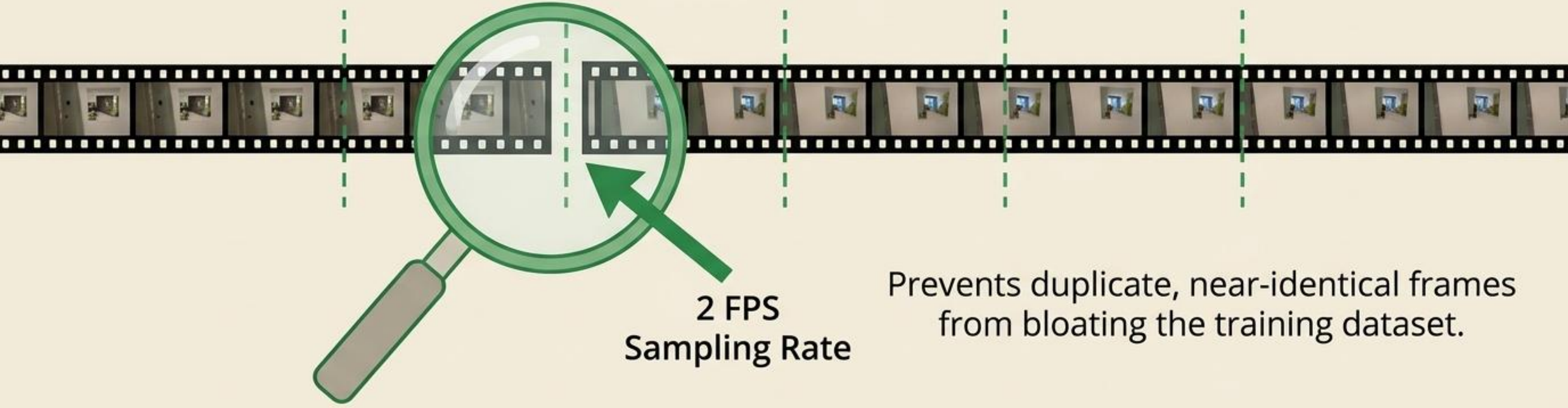
Encoded Target:	**30 FPS**
Actual ROS2 Publish Rate:	**~7.68 FPS**

ENGINEERING TAKEAWAY

Systems must be measured, not assumed. The 7.68 FPS bottleneck strictly dictates the downstream extraction sampling rate to prevent duplicate frames.

Gemini Notebook

FRAME SAMPLING PROCESS



Prevents duplicate, near-identical frames from bloating the training dataset.

VERIFICATION: frame_extractor.py output

```
$ python3 frame_extractor.py

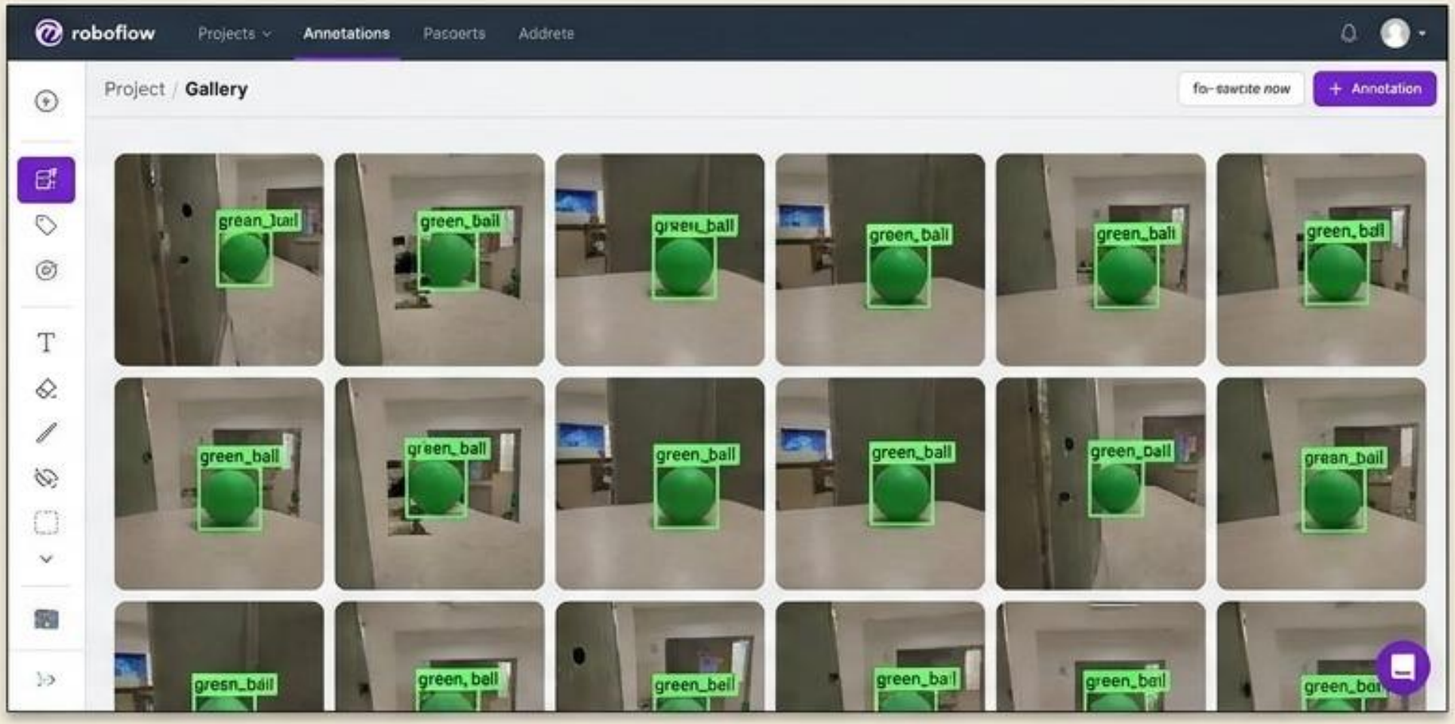
[INFO] Video processing complete.
353 JPEG images produced and saved to /data/raw_frames/
```

Extraction Parameters

```
Script: frame_extractor.py (Offline OpenCV)
Target_Rate: 2.0 Frames per Second
Output: 353 Raw JPEGs
```

Gemini Notebook

Cloud Annotation Directives (Roboflow)



Class Definition

Single target class designated as ‘green_ball’. Images resized to 640x640 (Stretch).

Negative Examples

Deliberately retain representative frames containing NO target ball. This establishes a baseline to reduce false positives.

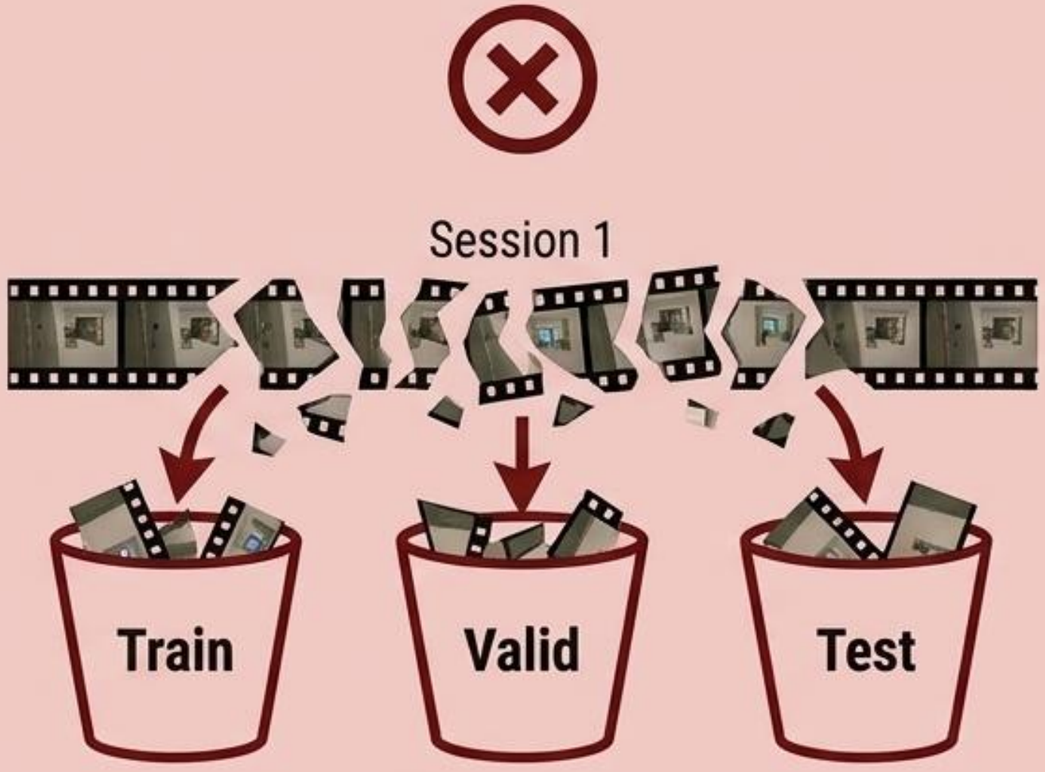
Hard Negatives

Do NOT label blue, yellow, or red balls. The model must learn the color distinction, not just spherical shapes.

Gemini Notebook

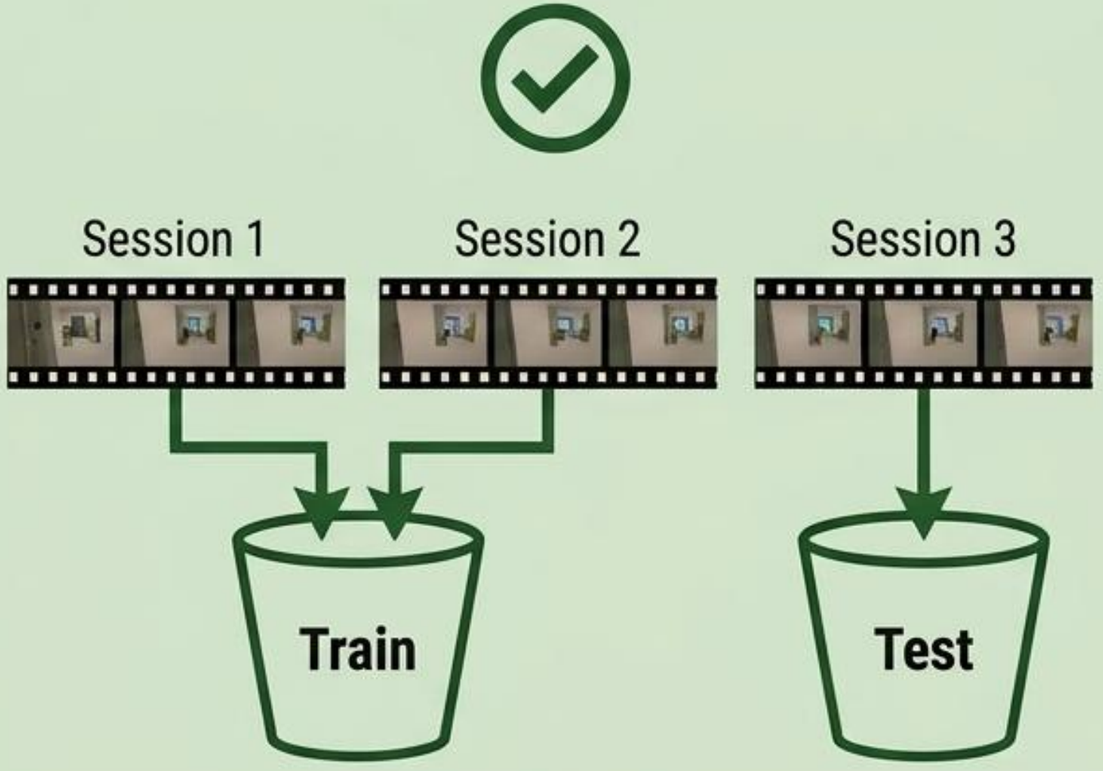
DATA SPLIT STRATEGY COMPARISON

The Flaw: Random Split



Data Leakage: High but fake validation scores. The model memorizes the background of the specific room instead of learning the object.

The Solution: Recording-Aware Split



True Independence: Ensures evaluation occurs on completely unseen environments and distinct lighting conditions.

Gemini Notebook

split_dataset.py

```python
import os
import shutil
import random

# Define source directories for each recording session
session_paths = {
    "session_1": "/path/to/session1/images",
    "session_2": "/path/to/session2/images",
    "session_3": "/path/to/session3/images"
}

# Define dictionary mapping sessions to splits
split_mapping = {
    "session_1": "train",   # Session 1: 166 images
    "session_2": "valid",   # Session 2: 144 images
    "session_3": "test"     # Session 3: 41 images (Held-Out Recording)
}

# Define output split directories
split_dirs = {
    "train": "/path/to/final_dataset/train",
    "valid": "/path/to/final_dataset/valid",
    "test": "/path/to/final_dataset/test"
}

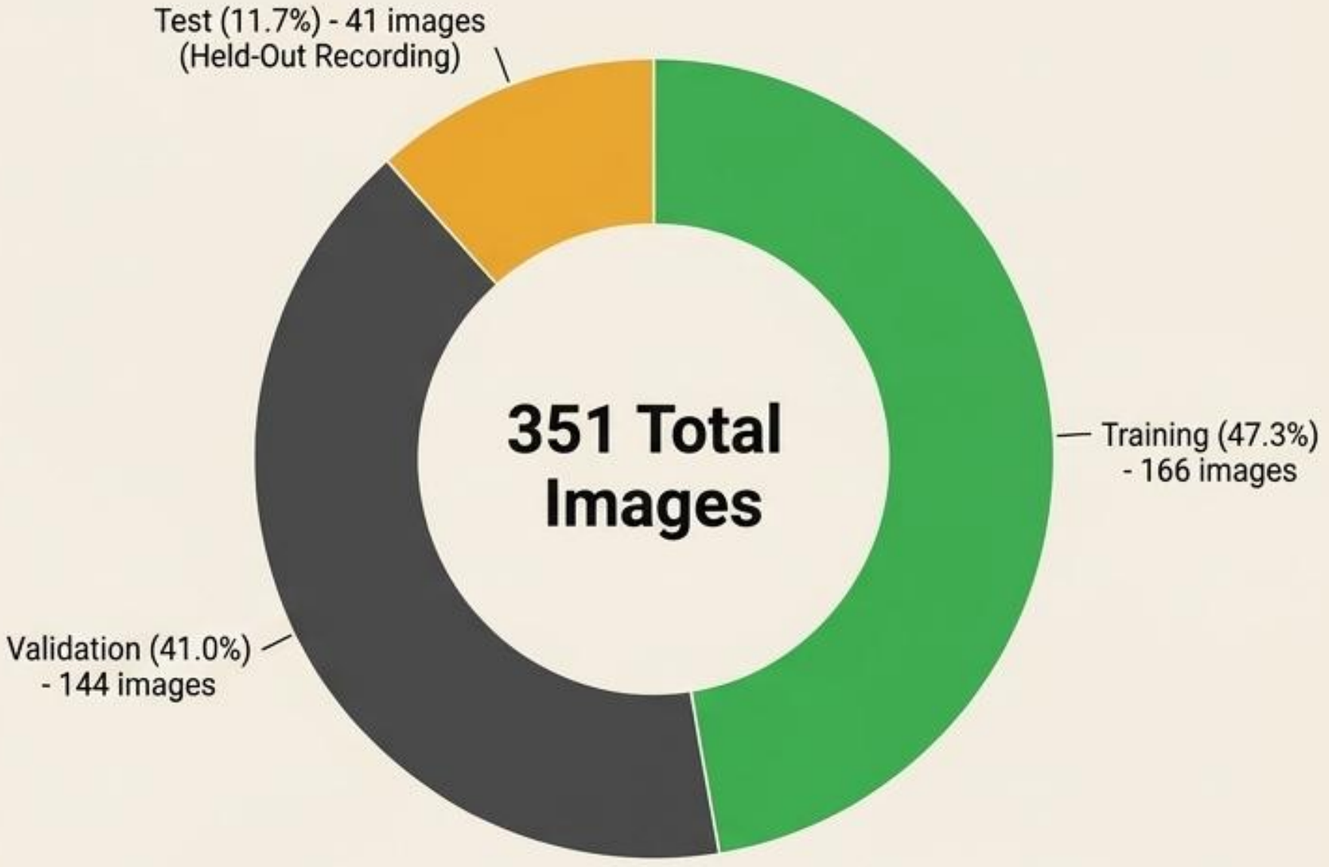
# Ensure output directories exist
for dir_path in split_dirs.values():
    os.makedirs(dir_path, exist_ok=True)

# Distribute images based on mapping
for session, split in split_mapping.items():
    source_dir = session_paths[session]
    destination_dir = split_dirs[split]
    for filename in os.listdir(source_dir):
        if filename.endswith('.jpg') or filename.endswith('.png'):
            src_file = os.path.join(source_dir, filename)
            dst_file = os.path.join(destination_dir, filename)
            shutil.copy2(src_file, dst_file)

print("Dataset splitting completed successfully based on recording sessions.")
```

Terminal × uocit

```
ilmi@ilmi:~/mindspark_ws$ python3 split_dataset.py
Dataset splitting completed successfully based on recording sessions.
ilmi@ilmi:~/mindspark_ws$ ls -l /path/to/final_dataset/train | grep "^-" | wc -l
166
ilmi@ilmi:~/mindspark_ws$ ls -l /path/to/final_dataset/valid | grep "^-" | wc -l
144
ilmi@ilmi:~/mindspark_ws$ ls -l /path/to/final_dataset/test | grep "^-" | wc -l
41
ilmi@ilmi:~/mindspark_ws$
```



Engineering Priority: Independence by recording session takes priority over achieving an exact 70/20/10 split.

Gemini Notebook

```
[sanna]-@yolo_venv: ~/pyTorch.runntans/nvdia cuda-apy
PyTorch CUDA detection: true
Thu Apr 14 02:018:181 2000
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 315.85.0     Driver Version: 415.85.0     CUDA Version: 11.2     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA RTX 2060 ..  On   | 00000000:16:00.0 Off |                  N/A |
| N/A   37C    PB    52W / 220W |     89MiB / 10528MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
```

GPU Hardware

NVIDIA RTX 2060

Compute Memory

6 GB VRAM

ML Framework

PyTorch 2.14.1+cu130

CUDA Enabled

Model Architecture

Ultralytics YOLO11

Version 8.4.183

LOCAL TRAINING MANDATE: Training occurs strictly locally on the robotics PC within 'yolo_venv' rather than a cloud service, ensuring total developer control over the robotics pipeline.

Gemini Notebook

YOLO11n Training & Evaluation Dashboard

Terminal: YOLO Training

```
(yolo_venv) [anna@yolo_venv:~/projects/robotics] $ yolo task=detect
mode=train model=yolo11n.pt data=data.yaml epochs=50 imgsz=640
Transferred 567/567 items from yolo11n.pt
...
Epoch    gpu_mem       box       obj       cls    labels  img_size
 50/50     3.42G   0.00557   0.00332   0.00174        19       640
...
Result saved to runs/detect/train
(yolo_venv) [anna@yolo_venv:~/projects/robotics] $ ls -lh runs/detect
/train/weights/best.pt
-rw-rw-r-- 1 anna anna 6.4M Apr 14 02:21 runs/detect/train/weights/be
st.pt
```

Training Parameters

Model: **YOLO11n** | Epochs: **50** | Input: **640x640**

Output Artifact: best.pt

Hardware Speed (Local RTX 2060)

Preprocess: **1.2ms** | Inference: **3.3ms** | Postprocess: **1.0ms**

Validation Metrics (Seen Sessions)

Precision: **0.999**
Recall: **1.000**
mAP50: **0.995**
mAP50-95: **0.908**

Independent Test Metrics (Unseen Session)

mAP50-95: **0.823**

Noticeable drop from validation (0.908) indicates a slight decrease in bounding box localization accuracy on the truly unseen recording.

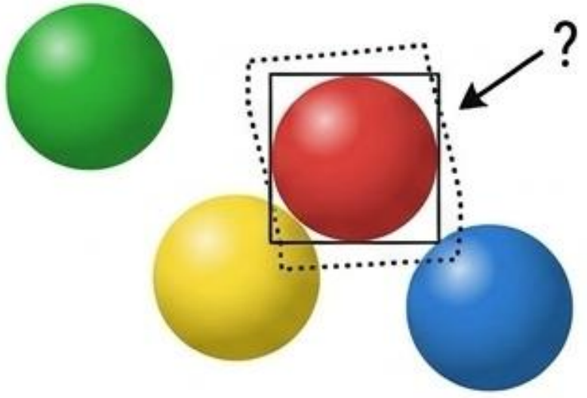
Gemini Notebook

THE EXPECTATION (METRICS)



Validation mAP50 of 0.995 implies near-perfect detection of the green ball in the training environments.

THE REALITY (LIVE INFERENCE)

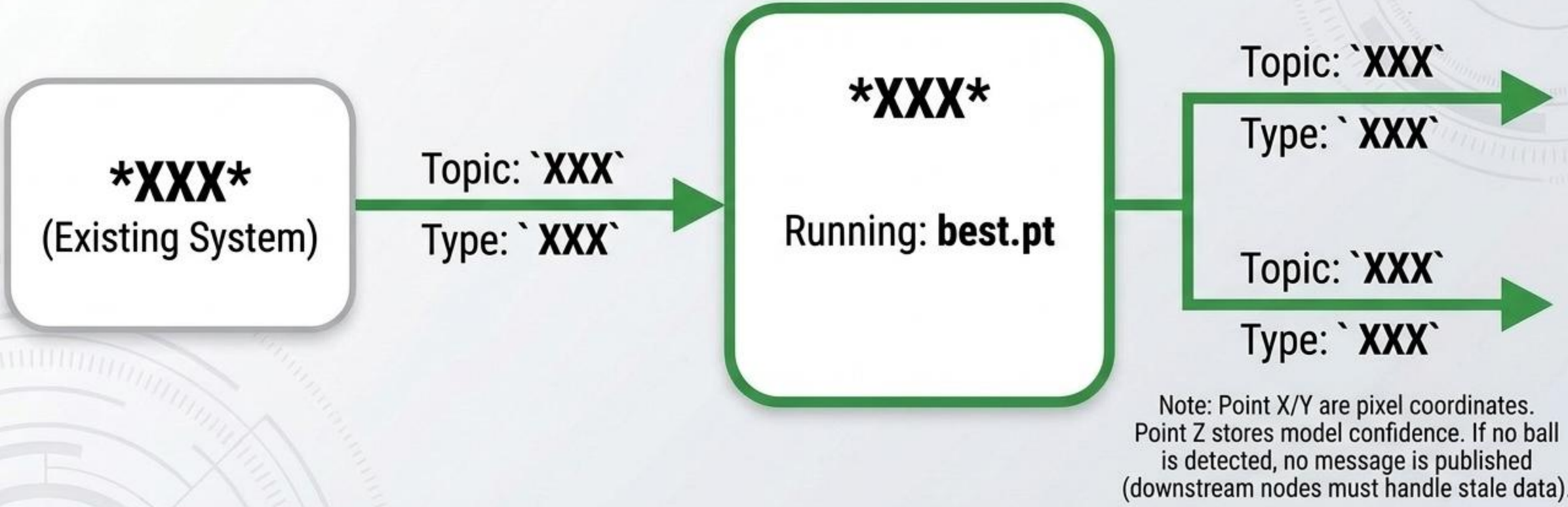


During live webcam challenges, the model occasionally predicts differently colored balls as 'green_ball'.

ENGINEERING CONCLUSION

High mAP50 proves excellent shape and boundary recognition, but the **limited dataset size prevents perfect color-specific generalisation.**
Next Step: Conduct **hard-negative mining** by feeding unlabelled frames of **red/blue/yellow balls** and **retrain.**

Gemini Notebook



Note: Point X/Y are pixel coordinates. Point Z stores model confidence. If no ball is detected, no message is published (downstream nodes must handle stale data).

Gemini Notebook

```
user@robot:~/ros2_ws$ python -m colcon build --pa
ckages-select mindspark_robot --symlink-install
[0.280s] WARNING:colcon.colcon_core.package_selec
tion:The following packages were skipped because
they are not in the workspace: mindspark_robot

Starting >>> mindspark_robot
[1.146s] [mindspark_robot] install/mindspark_robo
t/lib/mindspark_robo/mindspark_robot_node:
  #!/home/.../yolo_venv/bin/python
Finished <<< mindspark_robot [1.36s]

Summary: 1 package finished [1.58s]
```

THE INTEGRATION PROBLEM

Standard 'colcon build' points executables to '/usr/bin/python3'. This system Python lacks the ML dependencies (Ultralytics), causing 'ros2 run' to crash instantly.

THE VIRTUAL ENVIRONMENT SOLUTION

Override the build command to force ROS2 to use the active virtual environment interpreter.

```
python -m colcon build --packages-select
mindspark_robot --symlink-install
```

Verification: The resulting executable shebang must securely bind the node to the AI dependencies:

```
#!/home/.../yolo_venv/bin/python
```

Live Deployment Parameters

QoS Profiling (Quality of Service)

- **Input Subscription**: Uses `qos_profile_sensor_data` (Best effort). Drops frames if the network is busy to avoid lag build-up.
- **Output Publisher (Viewer)**: Upgraded to `RELIABLE` (Queue depth 10) to ensure the annotated image stream reaches the remote viewer without packet loss.

Live Telemetry & Display

- **Embedded Viewer**: The node utilizes an embedded `cv2.imshow` window for lightweight, direct viewing without requiring separate `image_tools` nodes.
- **Live Throughput**: The final live annotated output strictly matches the hardware camera bottleneck, operating stably at ~7.6 FPS.

Gemini Notebook

Engineering Insights: Root-Cause Analysis

Symptom	Root Cause	Solution / Insight
System encoded for 30 FPS, but inference runs at 7.6 FPS.	Hardware bottleneck. Encoded parameters differ from actual ROS2 publishing bandwidth.	Measure actual hz. Cap offline frame extraction rate to match hardware reality.
Excellent validation scores, but poor real-world testing.	Data Leakage. Random splitting put adjacent video frames into train and test sets.	Programmatic recording-aware splitting. Keep video sessions intact.
'ros2 run' crashes citing missing 'ultralytics' module.	Executable defaulted to system Python instead of the ML virtual environment.	Build using 'python -m colcon build' inside 'yolo_venv'.
False positives detecting non-green balls.	High mAP values reflect shape localization, lacking distinct color discrimination.	Expand dataset via hard-negative mining (unlabelled red/blue frames) and retrain.

Gemini Notebook